

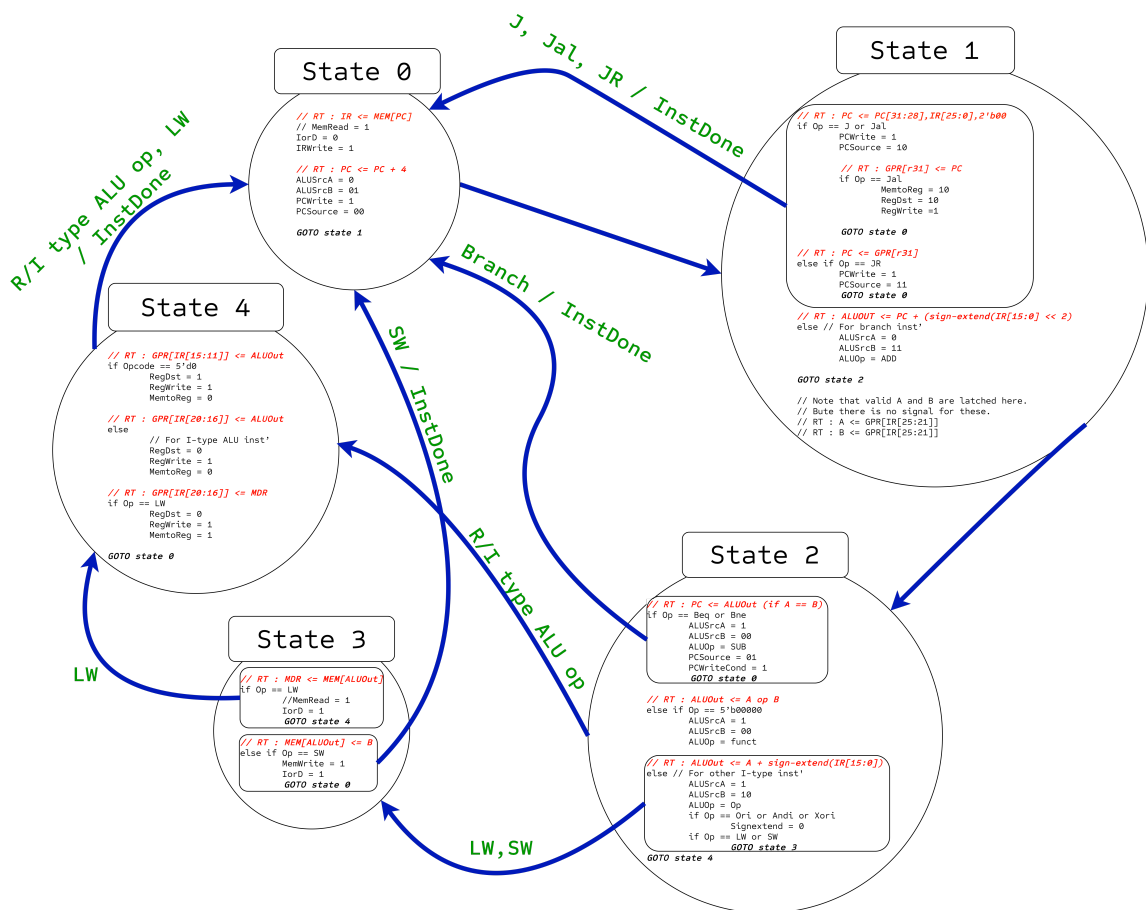
lab4_report

: 2022056562 서민용, 2023003227 이한주

1. Overall structure

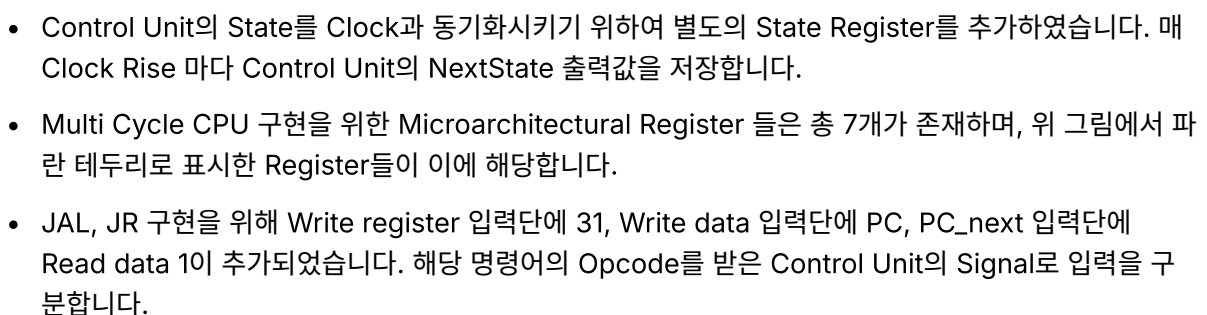
1-1. CTRL Logic; i.e. State Graph

- lab4는 기존 코드들을 활용하는 부분들도 많았고, signal에 대한 자유도가 많았습니다.
- 하여 코드를 작성하기 전, 과제 명세에 나온대로 State transfer logic을 Pseudo Code 형태로 작성한 후, 이를 State graph 형태로 나타냈습니다.



- 수업에서 다루었던 것처럼, 각 instruction 들에 대해 필요한 RT sequences 를 정리하고, Dependency가 없는 선에서 최대한 하나의 State 안에 몰아 넣을 수 있게끔 작성했습니다.
- JR과 JAL instruction 에 대해선 크게 어렵지 않았던 이유는,
 - 필요한 Signal 들의 정보만 다를 뿐이지 사실 상 두 instruction 모두 IF, ID 두 stage 만에 끝나기 때문에
 - 기존의 Jump inst와 동일하게 State transfer를 작성하였습니다.

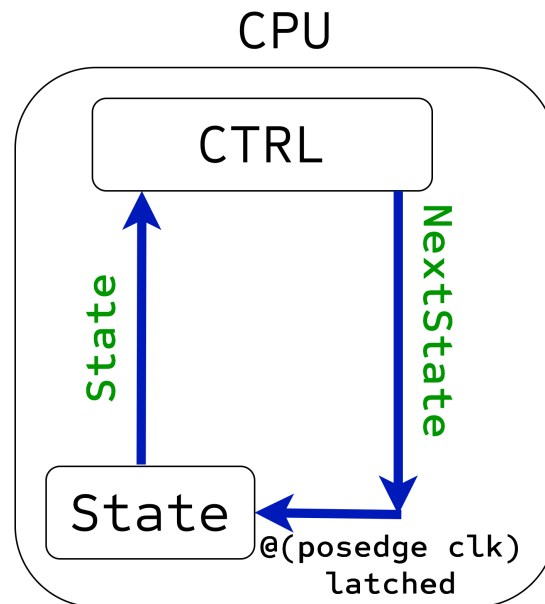
- ## 1-2. CPU Microarchitecture



2. Implementation

2-1. CTRL.v

- 기본적으로 CTRL logic은 Combinational Logic Unit 이라고 할 수 있습니다.
- 하지만 하나의 State 마다 한 Cycle 이 소요되기 때문에, state transfer과 cycle 주기를 맞추어주기 위해서 다음과 같은 방식을 채택했습니다.



- CTRL 은 clk 를 직접 input으로 받지는 않지만,
- TOP module 로서 작동하는 CPU에서 `always @(posedge clk)` 를 통해서 매 cycle 마다 State 값을 NextState로 업데이트 해주고,
- 이 State 값은 CTRL의 input으로 들어가게 되어 매 cycle 마다 state transfer가 일어나면서도, CTRL을 combinational 하게 구현하였습니다.
- State Graph를 그릴 때 작성한 Pseudo code를 바로 Verilog로 작성하면 되었기에 CTRL.v 코드 전체 내용을 따로 첨부하지 않았습니다.
- 다만 Compaction을 위해서, Branch instruction에 대한 처리가 비직관적으로 작성된 한 부분이 있습니다.
 - 우선 CPU.v 를 구현할 때 있어서, 최우선적으로 생각했던 것은 Data Path Diagram을 그대로 Code level 로 옮겨오자는 것이었습니다.
 - 하여 기존 branch에 대한 구현 역시 `... && (alu_result == 0)` 이라는 조건문을 사용했습니다.
 - 이는 Diagram에서 ALU Unit 에 있는 Zero 값을 그대로 AND Gate에 넣도록 명시되어 있기 때문입니다.
 - 혹은 더 원초적으로 생각했을 때, Branch condition Taken/NotTaken 에 대해서는, 두 Operand의 Subtraction이 0이라면 Taken, otherwise NotTaken 이라는 로직입니다.
 - 하나 이를 Subtraction을 이용해 Verilog code 에서 작성하려면, CPU.v 에서
 - `(PCWrite || (PCWriteCond && ((opcode == OP_BEQ) && alu_result == 0) || ((opcode == OP_BNE) && alu_result != 0)))` 이라는 조건문을 사용해야 합니다.

- 이는 너무 길고 보기에다 깔끔하지 않습니다.
- 혹은, GLOBAL.v 에서 다음과 같이 정의하고 있기에 bitwise 하게 최적화 하면 `(PCWrite || (PCWriteCond && (~opcode[0]^(!alu_result))))` 가 될 수 있습니다.

```
`define      OP_BEQ          6'd4
`define      OP_BNE          6'd5
```

- **하나 애초에 TOP module인 CPU.v 에서 opcode의 값을 읽고, 이에 따라 조건문을 다르게 해 준다?** 라는 것은, Reality 와 동떨어진 code 이기도 하고,
- 무엇보다 앞서 언급했듯 저희가 결정한 Data Path, CPU Microarchitecture와는 맞지 않습니다.
 - 단순 Bit Slicing/concatenation이 아니라, Bitwise XOR, Bitwise reduction OR operation이기에 Data path Diagram에서 통치고 넘어갈 수도 없습니다.
- 하여 `... && (alu_result == 0)` 을 그대로 유지하면서 구현하기 위해 다음과 같이 코드를 작성했습니다.

```
// CTRL.v
// ...
`STATE2: begin
    if (opcode == `OP_BEQ || opcode == `OP_BNE) begin
        ALUSrcA = 1;
        ALUSrcB = 2'b00;
        ALUOp = (opcode == `OP_BEQ) ? `ALU_NEQ : `ALU_E
Q;

        PCSource = 2'b01;
        PCWriteCond = 1;
        InstDone = 1;
        NextState = `STATE0;
    end
// ...

// CPU.v
if (PCWrite || (PCWriteCond && (alu_result == 0))) begin
    PC <= PC_next;
end
```

- opcode가 Beq 라면 ALUOp 에 ALU_NEQ 를 넣는 것이 조금 비직관적이지만, ALU.v의 logic을 보았을 때 완벽하게 똑같이 작동합니다.
- 결론적으로는 **PC <= ALUOut (if A == B)** 이라는 조금 Compact한 표현을 사용한 RT sequence을 나타내야하고, Data Path Diagram과의 일치성을 위해서 Code level에서의 조금의 고려사항이 있다고 볼 수 있습니다.
 - CPU.v 에 대해서는 Mux와 latch 등에 대해서 Data Path 그대로를 Code level로 옮겨온 logic이고,

- CTRL.v 는 Data Path에서 사실상 내부 로직이 정확히 명시되지 않은 Black Box이고, 저희만의 State Graph를 만들어 작성하도록 과제 명세에 나와있기에 위와 같이 CTRL.v 에서 다소 비직관적으로 작성했습니다.

2-2. CPU.v

- 1-2에 표시된 Multi Cycle CPU의 다이어그램을 기반으로 작성하였으며, Single Cycle CPU에서의 가장 큰 차이점은 Microarchitectural State를 저장하는 레지스터가 존재한다는 것입니다.
- 해당하는 7개의 레지스터는 Clk과 동기화되며, PC와 IR은 Enable Signal의 영향 또한 받습니다.
- 초기값 설정을 위해 rst Signal을 입력받은 경우 모든 레지스터의 값을 초기화해줍니다.

```
always @(posedge clk) begin
    if (rst) begin
        PC <= 0;
        IR <= 0;
        MDR <= 0;
        A <= 0;
        B <= 0;
        ALUOut <= 0;
        State <= `STATE0;
    end
    else begin
        if (PCWrite || (PCWriteCond && (alu_result == 0))) begin
            PC <= PC_next;
        end
        if (IRWrite) begin
            IR <= mem_read_data;
        end
        // always latched
        MDR <= mem_read_data;
        A <= rd_data1;
        B <= rd_data2;
        ALUOut <= alu_result;
        State <= NextState;
    end
end
```

- 나머지 MUX는 Combinational Logic이므로 해당하는 Signal에 맞게 올바른 Output을 내보내도록 작성했습니다.

```
always @(*) begin
    Address = IorD ? ALUOut : PC;
    case (RegDst)
        2'd0: wr_addr = rt;
```

```

        2'd1: wr_addr = rd;
        2'd2: wr_addr = 5'd31;
    endcase
    case (MemtoReg)
        2'd0: wr_data = ALUOut;
        2'd1: wr_data = MDR;
        2'd2: wr_data = PC;
    endcase
    Operand1 = ALUSrcA ? A : PC;
    case (ALUSrcB)
        2'd0: Operand2 = B;
        2'd1: Operand2 = 32'd4;
        2'd2: Operand2 = SignExtend ? {{16{immi[15]}}}, immi} : {1
6'b0, immi};
        2'd3: Operand2 = {{16{immi[15]}}}, immi} << 2;
    endcase
    case (PCSource)
        2'd0: PC_next = alu_result;
        2'd1: PC_next = ALUOut;
        2'd2: PC_next = (PC & 32'hF0000000) | (immj << 2);
        //2'd3: PC_next = A;
        2'd3: PC_next = rd_data1;
    endcase
end

```

3. Result & Inspection

- 모든 테스트케이스에 대해서 잘 통과한 모습입니다.

```

minyongseo@MinYong-MacBook-Air HW04_HanJoo % iverilog CPU.v CTRL.v ALU.v RF.v MEM.v CPU_tb.v GLOBAL.v
minyongseo@MinYong-MacBook-Air HW04_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 24895000 (1ps)
minyongseo@MinYong-MacBook-Air HW04_HanJoo %

```

```

minyongseo@MinYong-MacBook-Air HW04_HanJoo % iverilog CPU.v CTRL.v ALU.v RF.v MEM.v CPU_tb.v GLOBAL.v
minyongseo@MinYong-MacBook-Air HW04_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 7425000 (1ps)
minyongseo@MinYong-MacBook-Air HW04_HanJoo %

```

```

minyongseo@MinYong-MacBook-Air HW04_HanJoo % iverilog CPU.v CTRL.v ALU.v RF.v MEM.v CPU_tb.v GLOBAL.v
minyongseo@MinYong-MacBook-Air HW04_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 5405000 (1ps)
minyongseo@MinYong-MacBook-Air HW04_HanJoo %

```

```

minyongseo@MinYong-MacBook-Air HW04_HanJoo % iverilog CPU.v CTRL.v ALU.v RF.v MEM.v CPU_tb.v GLOBAL.v
minyongseo@MinYong-MacBook-Air HW04_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 27885000 (1ps)
minyongseo@MinYong-MacBook-Air HW04_HanJoo %

```

```

minyongseo@MinYong-MacBook-Air HW04_HanJoo % iverilog CPU.v CTRL.v ALU.v RF.v MEM.v CPU_tb.v GLOBAL.v
minyongseo@MinYong-MacBook-Air HW04_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 12595000 (1ps)
minyongseo@MinYong-MacBook-Air HW04_HanJoo %

```

```

minyongseo@MinYong-MacBook-Air HW04_HanJoo % iverilog CPU.v CTRL.v ALU.v RF.v MEM.v CPU_tb.v GLOBAL.v
minyongseo@MinYong-MacBook-Air HW04_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 6145000 (1ps)
minyongseo@MinYong-MacBook-Air HW04_HanJoo %

```

```

minyongseo@MinYong-MacBook-Air HW04_HanJoo % iverilog CPU.v CTRL.v ALU.v RF.v MEM.v CPU_tb.v GLOBAL.v
minyongseo@MinYong-MacBook-Air HW04_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 20335000 (1ps)
minyongseo@MinYong-MacBook-Air HW04_HanJoo %

```

```

minyongseo@MinYong-MacBook-Air HW04_HanJoo % iverilog CPU.v CTRL.v ALU.v RF.v MEM.v CPU_tb.v GLOBAL.v
minyongseo@MinYong-MacBook-Air HW04_HanJoo % ./a.out
VCD info: dumpfile waveform.vcd opened for output.
Total Cycles : 2033
Total Instructions: 500
IPC : 0.249877
CPI : 4.081969
Avg Time/Inst : 40.019685 ns
Program Terminate

Simulation success!!!
CPU_tb.v:60: $finish called at 20335000 (1ps)

```

- 또한 구현상 InstDone Signal은 아무 역할을 하지 않습니다. 하여 InstDone을 활용해보고자

- 7번 테스트케이스에 대해서만 다음과 같은 코드를 추가하여 performance inspection을 해보았습니다.

```
// CPU.v
// For performance measure
    reg [31:0] cycle_count;
    reg [31:0] inst_count;

// ...
initial begin
    cycle_count <= 0;
    inst_count <= 0;
end

always @(*) begin
    if(halt && (cycle_count > 10)) begin
        $display(" Total Cycles      : %0d", cycle_count);
        $display(" Total Instructions: %0d", inst_count);
        $display(" IPC                      : %f", $itor(inst_count) /
$itor(cycle_count));
        $display(" CPI                      : %f", $itor(cycle_count)
/ $itor(inst_count));
        $display(" Avg Time/Inst          : %f ns", ($itor(cycle_cou
nt) * 10.0) / $itor(inst_count));
    end
end

always @(posedge clk) begin
    cycle_count <= cycle_count + 1;
    if (InstDone) begin
        inst_count <= inst_count + 1;
    end
    // ...
end
```

- 물론 한 Cycle 이내에 모든 연산이 이루어진다는 매우 Naive한 환경과,
- SingleCycle 과 MultiCycle의 Cycle 길이 : 10ns 에 대해서 적절한 예상값으로라도 변경해주지 않았음을 고려하면 큰 의미가 있지는 않습니다.
- Multicycle에 대해서 CPI가 대략 4정도 나오고, testbench에서의 init 시간까지 포함하여 걸린 시간은 총 20335000ps 입니다.
- lab3_report에서 작성했듯 SingleCycle에 대해서는 testbench에서의 init 시간까지 포함하여 걸린 시간은 총 5085000ps 입니다.
- 이렇듯 Naive 하게 연산하였을 때 4배 정도의 성능 향상이 있음을 알 수 있습니다.

4. Troubleshooting

- 초기에 ZeroExtend에 대한 고려를 하지 않고 구현했었습니다. 이는 CPU Data Path를 완벽하게 결정하지 못했기 때문인데,
 - ALUSrcB를 3 bit signal로 확장하여 ALUSrcB 내부에서 자체적으로 ZeroExtend 여부를 확인할 것인지,
 - 혹은 SignExtend Signal 사용할 것인지 결정하지 못했었기 때문입니다.
- 하여 ZeroExtend를 사용하지 않는 테스트케이스에 대해서만 통과시키고, 추후 결정하기로 하였으나, **이상하게도 ZeroExtend를 사용하지 않는 테스트케이스에 대해서도 실패를 하였습니다.**
- Debugging의 간결함을 위해서, 제공된 5번째 테스트케이스의 크기를 매우 줄여서 돌려보고 이를 MARS에서 비교를 해보았더니, 다소 특이한 점을 발견했습니다.

lab4 > mytestcase > ~~asm~~ mips1.asm

```
1 .text
2 main:
3     li $t0, -1588822599
4     li $t1, 1284959778
5     li $t2, -2063573906
6     li $t3, 90508380
7     li $t4, 1432376667
8     li $t5, -1755702711
9     jal func
10    j exit
11 func:
12    li $s0, 1938814828
13    li $s1, 338139922
14    li $s2, -1743822069
15    li $s3, 461933792
16    li $s4, -1699543183
17    li $s5, 575516400
18    jr $ra
19 exit:
20    li $v0, 5
21
```

Debugging :

	Value
index: 0, dat: 00000000	0 0x00000000
index: 1, dat: 224d0000	1 0x224d0000
index: 2, dat: 00000005	2 0x00000005
index: 3, dat: 00000000	3 0x00000000
index: 4, dat: 00000000	4 0x00000000
index: 5, dat: 00000000	5 0x00000000
index: 6, dat: 00000000	6 0x00000000
index: 7, dat: 00000000	7 0x00000000
index: 8, dat: a14c7db9	8 0xa14c7db9
index: 9, dat: fffffee2	9 0x4c96ee22
index: 10, dat: 85005c6e	10 0x85005c6e
index: 11, dat: 05650c5c	11 0x05650c5c
index: 12, dat: 5560555b	12 0x5560555b
index: 13, dat: 975a1a49	13 0x975a1a49
index: 14, dat: 00000000	14 0x00000000
index: 15, dat: 00000000	15 0x00000000
index: 16, dat: fffff76c	16 0x738ff76c
index: 17, dat: ffff9b12	17 0x14279b12
index: 18, dat: 980f630b	18 0x980f630b
index: 19, dat: ffff8ce0	19 0x1b888ce0
index: 20, dat: 9ab30771	20 0x9ab30771
index: 21, dat: ffffaef0	21 0x224daef0
index: 22, dat: 00000000	22 0x00000000
index: 23, dat: 00000000	23 0x00000000
index: 24, dat: 00000000	24 0x00000000
index: 25, dat: 00000000	25 0x00000000
index: 26, dat: 00000000	26 0x00000000
index: 27, dat: 00000000	27 0x00000000
index: 28, dat: 00001800	28 0x00001800
index: 29, dat: 00003ffc	29 0x00003ffc
index: 30, dat: 00000000	30 0x00000000
index: 31, dat: 00000034	31 0x00000034

reduced testcase5 / Verilog RF / MARS RF

- 특정 Register에 대해서, 정확히 MSB 16bit가 FFFF 이거나, 그렇지 않고 다른 값이냐라는 점을 발견했습니다.
- 또한 수업시간에 다루었듯 32bit system에서는, 64bit immediate 를 load 하기 위해서 lui와 ori로 나누어 처리한다는 것을 깨달았습니다.
 - 즉, li inst 역시 ZeroExtend를 사용하는 연산에 간접적으로 포함되는 것을 인지하고, 곧바로 ZeroExtend를 구현해주었더니 모든 테스트케이스가 맞게 돌아갔습니다.